HOWARD Help Calculations

1 HOWARD Calculations

List of available calculations in HOWARD. For each calculation, the name, description, and comment are provided. The comment may include additional information, examples, or references to help understand the calculation and its usage.

See Parameters JSON help for more details on how to specify calculation parameters in the JSON parameter file.

1.1 ANNOTATION

Annotation of variants based on specified databases and tools

Description

Annotate variants based on specified databases and tools. This calculation allows for the annotation of variants using parameters specified in the JSON parameter file in the annotation parameters (see 'Annotation section' in help.parameters.pdf), including options on tools to use (HOWARD parquet algorithm, Annovar, snpEff, BCFTools...). Example that annotate variants with database in parquet format, annovar, snpEff and BCFTools with databases in VCF or BED format:

```
{
   "annotation": {
      "parquet": {
         "annotations": {
            "my.database.parquet": {...},
            "my.other.database.parquet": {...}
         }
      },
      "annovar": {...}
      "snpeff": {...}
      "bcftools": {...}
   }
}
```

1.2 ANNOTATION_EXTRACT

Extract annotation from structured INFO field (e.g. snpEff, VEP...)

Description

Extract annotation and HGVS nomenclatures from structured annotation and create new INFO fields with a prefix (e.g. 'snpeff_' for 'snpeff_hgvs', 'snpeff_impact', 'vep_gene_name'...). This calculation parses the annotation field from an annotation (e.g. 'ANN' for snpEff, 'CSQ' for VEP), extracting relevant information such as HGVS nomenclatures, impact, gene name, etc., and explode new INFO fields with the appropriate prefix for easier access and downstream analysis.

Example with snpEff annotation (field ANN), extract HGVS nomenclature, explode information with prefix 'snpeff_',and create a JSON field 'snpeff_json':

```
{
    "annotation_field": "ANN",
    "annotation_hgvs": "snpeff_hgvs",
    "annotation_explode": "snpeff_",
    "annotation_json": "snpeff_json",
    "uniquify": false
}
```

Example with VEP annotation (field CSQ), extract HGVS nomenclature, explode information with prefix 'vep_', and create a JSON field 'vep_json':

```
{
    "annotation_field": "CSQ",
    "annotation_hgvs": "vep_hgvs",
    "annotation_explode": "vep_",
    "annotation_json": "vep_json",
    "uniquify": false,
    "annotation_id": "Feature",
    "hgvs_columns": {
       "gene": "Gene",
       "transcript": "Feature",
       "rank": "EXON",
       "HGVSc": "HGVSc",
       "HGVSp": "HGVSp"
    },
    "hgvs_columns_contain_transcript_id": True
}
```

Example with VEP annotation (field CSQ), extract HGVS nomenclature using RefSeq (specific HGVS identifier column 'SYMBOL'), explode information with prefix 'vep_refseq_', and create a JSON field 'vep_refseq_json':

```
{
    "annotation_field": "CSQ",
    "annotation_hgvs": "vep_refseq_hgvs",
    "annotation_explode": "vep_refseq_",
    "annotation_json": "vep_refseq_json",
    "uniquify": false,
    "annotation_id": "Feature",
    "hgvs_columns": {
       "gene": "SYMBOL",
       "transcript": "Feature",
       "rank": "EXON",
       "HGVSc": "HGVSc",
       "HGVSp": "HGVSp"
    },
    "hgvs_columns_contain_transcript_id": True
}
```

1.3 BARCODE

BARCODE as VaRank tool

Description

BARCODE as VaRank tool. This calculation generates a BARCODE for each sample based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate).

Example of barcode calculation parameters for a variant with genotypes:

```
{
   "tag": "barcode",
   "tag_description": "BARCODE as VaRank tool"
}
```

1.4 BARCODE_FAMILY

BARCODE_FAMILY as VaRank tool

Description

BARCODE_FAMILY as VaRank tool. This calculation generates a BARCODE for each family based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate) and family relationships (within genotype information on

each sample).

Example of barcode family calculation parameters for a list of samples as a family:

```
{
   "tag": "BCF",
   "tag_description": "BARCODE_FAMILY as VaRank tool"
   "tag_samples": "BCFS",
   "tag_samples_description": "Description of tag samples",
   "family_pedigree": "sample1,sample2,sample4"
}
```

Example of barcode family calculation parameters for a list of samples in a JSON file:

```
{
   "tag": "BCF",
   "tag_description": "BARCODE_FAMILY as VaRank tool"
   "tag_samples": "BCFS",
   "tag_samples_description": "Description of tag samples",
   "family_pedigree": "path/to/family_pedigree.json"
}
```

With family_pedigree.json content example:

```
{
   "father": "sample1",
   "mother": "sample2",
   "child": "sample4"
}
```

1.5 EXPORT_STATS

Export statistics of variants to a file

Description

Export statistics of variants to a JSON file. Define the output file and whether to include annotation statistics. See 'Stats section' in Parameters JSON Help for more information.

Example of a statistics files (Markdown and JSON format) including annotation statistics and two queries:

```
{
   "stats_md": "my.stats.pdf",
   "stats_json": "my.stats.json",
   "annotations_stats": true,
   "queries": {
      "First 10 variants": "SELECT \"#CHROM\", POS, REF, ALT FROM variants_view LIMIT 10",
      "First 10 INFO tags": "SELECT INFOS.* FROM variants_view LIMIT 10"
   }
}
```

1.6 EXPORT_VARIANTS

Export variants to a file

Description

Export variants to a VCF file. Define the output file and format will depend on the extension. Options to explode fields and export them are available. See Parameters JSON Help for more information.

Example of a variant file in VCF (compressed):

```
{
   "file": "my.variants.vcf.gz",
}
```

Example of a variant file in TSV (uncompressed) with exploded fields, an header, and ordered by DP values, and filtered by specific query:

```
{
   "file": "my.variants.tsv",
   "explode": {
      "explode_infos": true,
      "explode_infos_prefix": "test_",
      "explode_infos_fields": ["CLNSIG", "SIFT", "DP"]
   },
   "export": {
      "fields_to_rename": {
         "CLNSIG": "CLNSIG_renamed",
         "SIFT": null
      },
      "order_by": "DP DESC",
      "include_header": true
   },
   "query": "SELECT * FROM variants WHERE INFO LIKE '%CLNSIG%'"
}
```

1.7 FIND_SAMPLES

Number of sample that have a genotype for the variant (for multi sample VCF)

Description

This calculation counts the number of samples that have a genotype for a given variant in a multi-sample VCF file. It helps in assessing the presence of the variant across different samples.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'tags' allows for the specification of the INFO tags to be created, with the key being the tag name and the value being either 'count' or 'list' to indicate whether to count the samples or list them. For example, to create an INFO tag 'count_samples' that counts the number of samples and an INFO tag 'list_samples' that lists the samples, you can specify:

```
{
   "tags": {
      "count_samples": "count",
      "list_samples": "list"
   }
}
```

Otherwise, change the default tags in the parameter file, like in this example:

```
{
   "tags": {
      "count_samples_for_variants": "count",
      "list_samples_for_variants": "list",
      "another_list_samples_tag": "list"
   }
}
```

1.8 GENOTYPE_CONCORDANCE

Concordance of genotype for multi caller VCF

Description

Concordance of genotype for multi caller VCF. This calculation assesses the concordance of genotypes for a given variant across multiple callers in a multi-caller VCF file. It helps in evaluating the consistency of genotype calls and can be used to identify variants with high confidence based on agreement among different callers.

Exemple of concordance calculation parameters for a variant with genotypes:

```
{
   "tag": "genotype_concordance",
   "tag_description": "Concordance of genotype for multi caller VCF"
}
```

1.9 GENOTYPE_STATS

Calculate genotype statistics on a specific genotype field (e.g. VAF, DP, GQ...)

Description

Calculate genotype statistics on a specific genotype field (e.g. VAF, DP, GQ...). This calculation computes statistical measures for a specified genotype field across different samples, providing insights into the distribution and variability of the chosen genotype metric.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'infos' specify the genotype fields (either a string or a list of strings) to be analyzed.

Example of VAF statistics calculation:

```
{
   "infos": "VAF"
}
```

Example of VAF and GQ statistics calculation:

```
{
   "infos": ["VAF", "GQ"]
}
```

If no 'infos' parameter is provided, the calculation will default to analyzing the VAF field. Other genotype fields (with Integer values) can be specified as needed, such as DP (Depth), GQ (Genotype Quality), etc., by providing the appropriate field name in the 'infos' parameter.

1.10 INFO_TO_FORMAT

INFO to FORMAT conversion

Description

INFO to FORMAT conversion. This calculation converts INFO fields to FORMAT fields in the VCF file.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf):

- 'annotation_fields': allows for the specification of the INFO fields to be converted to FORMAT fields, with the key being the INFO field name and the value being the FORMAT field name. For example, to convert INFO field 'count_samples' to FORMAT field 'CS', INFO field 'list_samples' to FORMAT field 'LS', and INFO field 'calling_quality' to FORMAT field 'CQ', you can specify:
- 'samples': allows for the specification of the samples to be included in the conversion. If not specified, all samples will be included.
- 'remove_info_fields': allows for the removal of the original INFO fields after conversion to FORMAT fields. If set to true, the original INFO fields will be removed from the VCF file.

Options example:

```
{
    "annotation_fields": {
        "count_samples": null,
        "list_samples": "LS",
        "calling_quality": "CQ"
    },
    "samples": ["sample1", "sample2"],
    "remove_info_fields": true
}
```

1.11 NOMEN

NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help)

Description

Extract NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature fields (see parameters help). This calculation parses the HGVS nomenclature fields to extract specific NOMEN information such as NOMEN, CNOMEN, PNOMEN, etc., based on the parameters provided. Multiple annotation fields are allowed (e.g. 'snpeff_hgvs', 'snpeff_hgvs,vep_hgvs'). It creates new INFO fields with the extracted NOMEN information for easier access and downstream analysis (i.e. 'NOMEN', 'CNOMEN', 'PNOMEN', etc.).

Example of NOMEN extraction from snpeff_hgvs field with default NOMEN patterns:

```
{
   "hgvs_fields": "snpeff_hgvs",
   "uniquify_hgvs": false,
   "nomen_pattern": "GNOMEN:TNOMEN:ENOMEN:CNOMEN:RNOMEN:NNOMEN:PNOMEN",
   "transcripts_column": "PZTTranscript",
   "transcripts": "my.transcripts.tsv",
   "transcripts_order": ["column", "file"],
}
```

1.12 PRIORITIZATION

Prioritization of variants based on specified profiles

Description

Prioritize variants based on specified profiles. This calculation allows for the prioritization of variants using parameters specified in the JSON parameter file in the prioritization parameters (see help.prioritization.pdf), including options on profiles to use and scoring methods. Example that prioritize variants using default and GERMLINE profiles:

```
{
   "prioritization": {
      "profiles": ["default", "GERMLINE"],
      "pzfields": ["PZFlag", "PZScore", "PZClass", "PZComment", "PZInfos", "PZTags"]
   }
}
```

1.13 RECREATE_INFO_FIELDS

Recreate INFO_tags, rename or remove tags

Description

Recreate INFO_tags, rename or remove tags. This calculation allows for the recreation of INFO fields by renaming existing tags or removing them based on specified parameters. It provides flexibility in managing INFO fields, enabling users to customize their VCF annotations according to their analysis needs.

This example will rename INFO field 'ENOMEN' to 'exon' and remove INFO fields 'SPiP', 'SPiP_Alt' and 'SPiP_distSS' from the 'variants' table:

```
{
   "fields_to_rename": {
      "ENOMEN": "exon",
      "SPiP": null,
      "SPiP_Alt": null,
      "SPiP_distSS": null
   },
  "table": "variants"
}
```

1.14 RENAME_INFO_FIELDS

Rename or remove INFO/tags

Description

Rename or remove INFO/tags. This calculation allows for the renaming of existing INFO fields or the removal of specific tags (use 'null') based on provided parameters. It helps in standardizing or customizing INFO fields in the VCF according to user requirements and facilitates downstream analysis. Use parameter section 'calculation_RENAME_INFO_FIELDS' in param.json to specify the INFO fields to rename or remove.

This example will rename INFO field 'ENOMEN' to 'exon' and remove INFO fields 'SPiP', 'SPiP_Alt' and 'SPiP_distSS' from the 'variants' table:

```
{
   "fields_to_rename": {
      "ENOMEN": "exon",
      "SPiP": null,
      "SPiP_Alt": null,
      "SPiP_distSS": null
   },
  "table": "variants"
}
```

1.15 TRANSCRIPTS_ANNOTATIONS

Perform transcripts annotations and generate a transcripts table/view (using JSON parameters file)

Description

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf), or directly in the calculation parameters.

Example that configure transcripts table with JSON parameters, defined by existing annotations on variants:

```
{
    "table": "transcripts",
    "transcripts_info_field_json": "transcripts_json",
    "transcripts_info_field_format": "transcripts_ann",
    "transcripts_info_json": "transcripts_json",
    "transcripts_info_format": "transcripts_format",
    "transcript_id_remove_version": true,
    "transcript_id_mapping_file": "transcripts.for_mapping.tsv",
    "transcript_id_mapping_force": false,
    "struct": {
    "from_column_format": [
            {
                "transcripts_column": "ANN",
                "transcripts_infos_column": "Feature_ID",
                "column_clean": true,
                "column_case": "lower"
            }
        ],
        "from_columns_map": [
            {
                "transcripts_column": "Ensembl_transcriptid",
                "transcripts_infos_columns": [
                    "genename",
                    "Ensembl_geneid",
                    "LIST_S2_score",
                    "LIST_S2_pred"
```

```
                ],
                "column_rename": {
                    "LIST_S2_score": "LISTScore",
                    "LIST_S2_pred": "LISTPred"
                },
            },
            {
                "transcripts_column": "Ensembl_transcriptid",
                "transcripts_infos_columns": [
                    "genename",
                    "VARITY_R_score",
                    "Aloft_pred
                ]
            }
        ]
    }
}
```

1.16 TRANSCRIPTS_EXPORT

Export transcripts table/view as a file (using JSON parameters file)

Description

Export transcripts table/view as a file. This calculation allows for the export of the transcripts table or view generated from transcript annotations to an external file format such as TSV or CSV. The parameters for this calculation can be specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), including options for the output file path, format, and any filters to apply during export.

Example that export transcripts table to a TSV file, with all INFO annotations, and supplementary exploded INFO fields:

```
{
   "export": {
       "output": "transcripts_export.tsv",
       "export_header": true,
       "add_info": true
   },
   "explode": {
       "explode_infos": true,
       "export_info_prefix": '',
       "explode_infos_fields": 'HGVS,.*_score,Clinvar'
   },
}
```

1.17 TRANSCRIPTS_PRIORITIZATION

Prioritize transcripts with a prioritization profile (using JSON parameters file)

Description

Prioritize transcripts with a prioritization profile. This calculation allows for the prioritization of transcripts based on a predefined profile specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), helping to identify the most relevant transcripts for further analysis.

Example that configure transcript prioritization with already existing transcripts table with JSON parameters:

```
{
    "prioritization": {
        "profiles": ["transcripts"],
        "prioritization_config": "config/prioritization_transcripts_profiles.json",
        "pzprefix": "PZT",
        "pzfields": ["Score", "Flag", "LISTScore", "LISTPred"],
        "prioritization_transcripts_order": {
```

```
            "PZTFlag": "DESC",
            "PZTScore": "DESC"
        },
        "prioritization_score_mode": "HOWARD",
        "prioritization_transcripts_file": null,
        "prioritization_transcripts_columns": null,
        "prioritization_transcripts_force": false,
        "prioritization_transcripts_version_force": false,
        "strict": false
    }
}
```

1.18 TRIO

Inheritance for a trio family

Description

Inheritance for a trio family. This calculation assesses the inheritance pattern of a variant within a trio family pedigree (father, mother, and child). It helps in understanding the transmission of genetic variants and identifying potential de novo mutations.

Example of trio calculation parameters for a variant with genotypes:

```
{
   "tag": "trio",
   "tag_description": "Inheritance for a trio family"
   "trio_pedigree": {
      "father": "sample_father",
      "mother": "sample_mother",
      "child": "sample_child"
   }
}
```

Options for these parameteres can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf).

The parameter 'trio_pedigree' allows for the specification of the sample names for father, mother, and child:

```
{
   "father": "sample_father",
   "mother": "sample_mother",
   "child": "sample_child"
}
```

This parameter can also be specified in a JSON file (containing the trio sample names):

```
 "/path/to/trio_samples.json"
```

If no pedigree information is provided, the calculation will use the first 3 samples in the VCF file.

1.19 VAF_NORMALIZATION

Variant Allele Frequency (VAF) normalization

Description

Variant Allele Frequency (VAF) normalization. This calculation normalizes the Variant Allele Frequency (VAF) across different samples and callers to ensure consistency in VAF representation. It helps in comparing VAF values across samples and callers by applying a harmonization process.

1.20 VARIANT_CHROM_POS_REF_ALT

Create a variant ID with chromosome, position, ref and alt, with format '#CHROM_POS_REF_ALT'

Description

This calculation generates a variant ID with chromosome, position, ref and alt, with format '#CHROM_POS_REF_ALT'. Useful for variant identification and comparison across datasets

1.21 VARIANT_FILTER

Filter variants based on specified criteria (using SQL parameters and sample list)

Description

Filter variants based on specified criteria. This calculation allows for the filtering of variants using SQL-like parameters specified in the JSON parameter file in the calculation parameters (see help.parameters.pdf), including options for the filtering conditions and any additional criteria (such as a list of samples) to apply during the filtering process. Example that filter variants on ClinVar pathogenic, on DP >= 100 for 'sample1', selection of only 2 samples, and ensure only not null genotypes:

```
{
   "filter_name": "Filter on ClinVar pathogenic, DP for sample1, select only 2 samples, and ensure only not n
   "where_clause": "INFOS.CLNSIG = 'pathogenic' AND SAMPLES.sample1.DP >= 100",
   "sample_list": ["sample1", "sample2"],
   "genotype_filter": true
}
```

1.22 VARIANT_ID

Variant ID generated from variant position and type

Description

Variant ID generated from variant position and variation (ref and alt) and type (SVTYPE). This calculation creates a unique identifier for each variant, facilitating variant tracking and comparison across datasets. Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'variant_id_tag' allows for the specification of the INFO tag to be used for the variant ID. By default, it uses 'variant_id', but it can be changed to any other INFO tag name as needed. For example, to use 'varid' as the INFO tag for the variant ID, you can specify:

```
{
   "variant_id_tag": "varid"
}
```

Other options as 'variant_id_tag_info' allows for the specification of the description in the VCF header, and 'keep_variant_id_tag_column' allows for keeping the variant ID tag column in the 'variants' table for further analysis.

As an axample of the JSON parameter file, you can specify:

```
{
   "variant_id_tag": "varid",
   "variant_id_tag_info": "Variant ID generated from variant position and type",
   "keep_variant_id_tag_column": true
}
```

1.23 VARTYPE

Variant type (e.g. SNV, INDEL, MNV, BND...)

Description

Determine the type of variant based on its characteristics, such as the length of the reference and alternate alleles, and the presence of structural variant information. This calculation classifies variants into types like SNV, INDEL, MNV, BND, etc., which is essential for downstream analysis and interpretation.